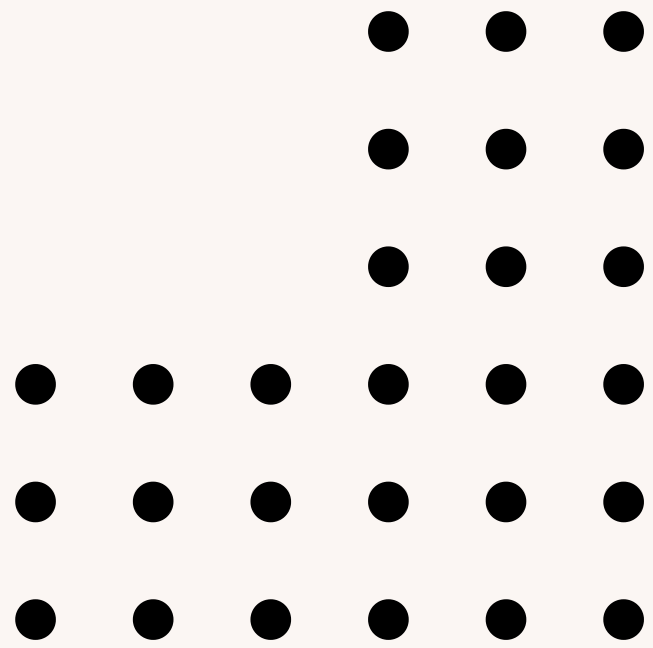
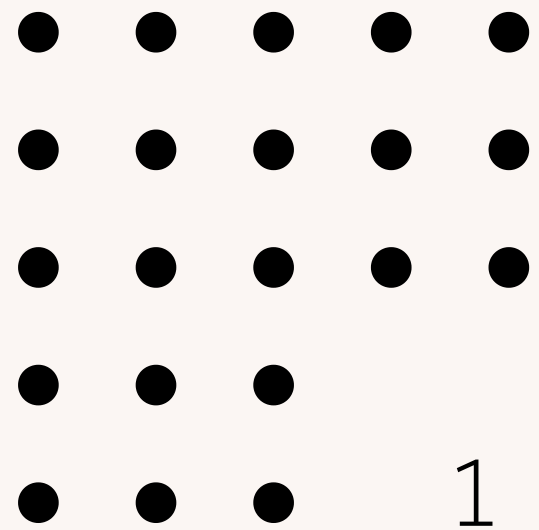
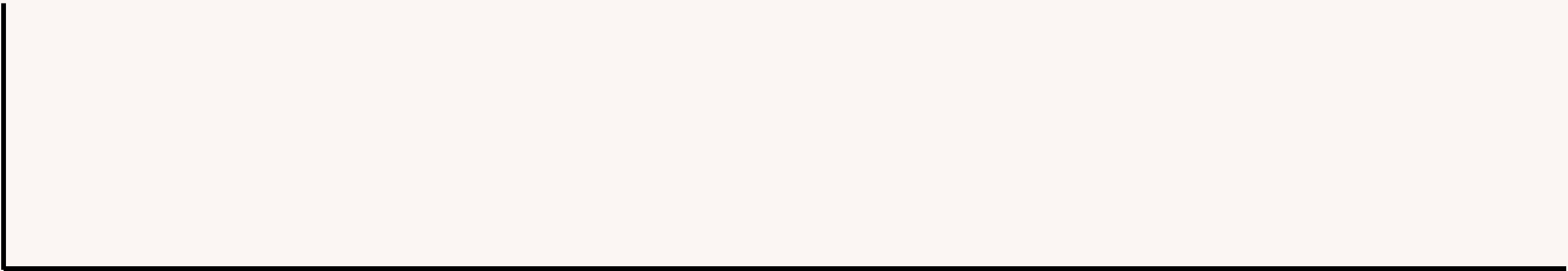
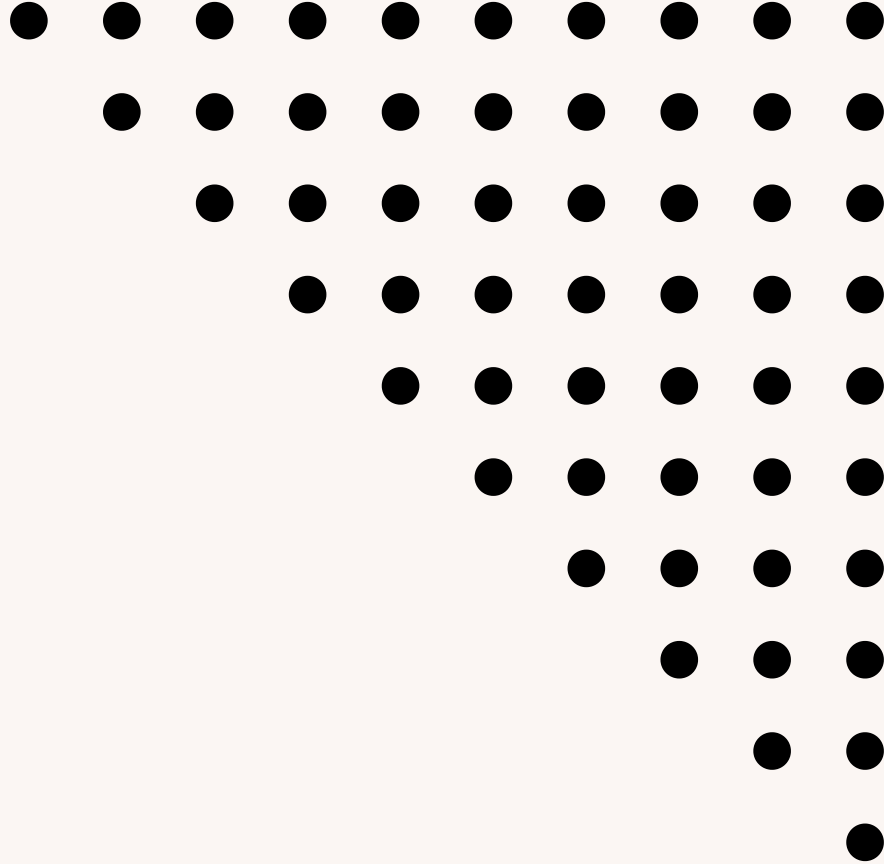


CLOTHING RECOMMENDATION SYSTEM

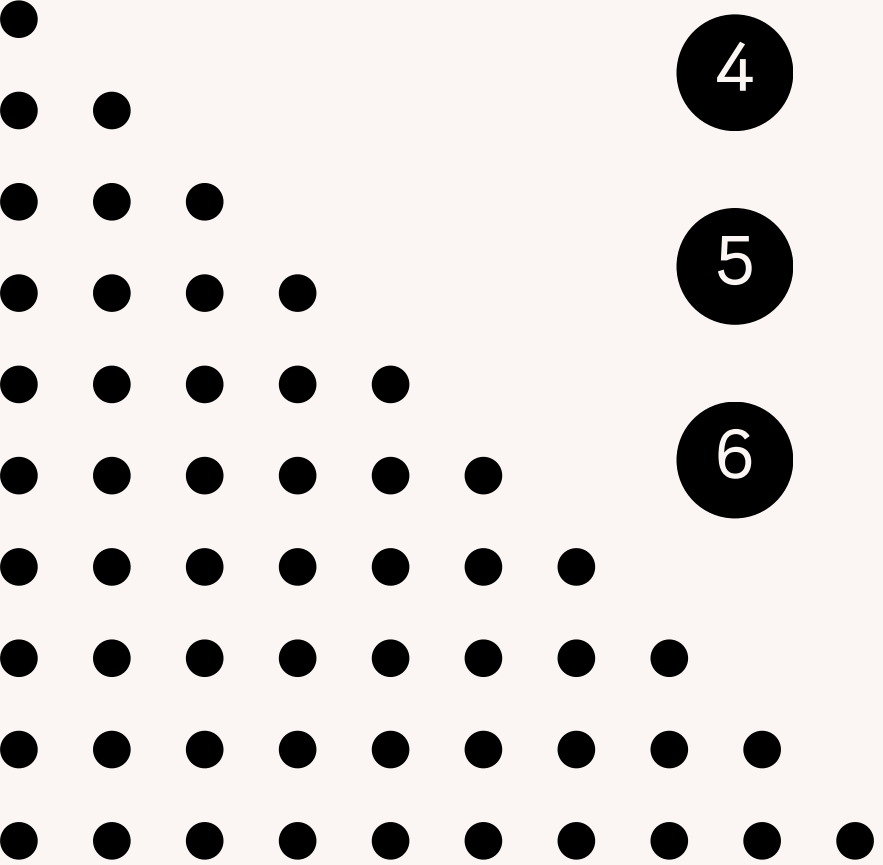
Ema Martins - up202402794

Tomás Ribeiro - up202007710





INDEX

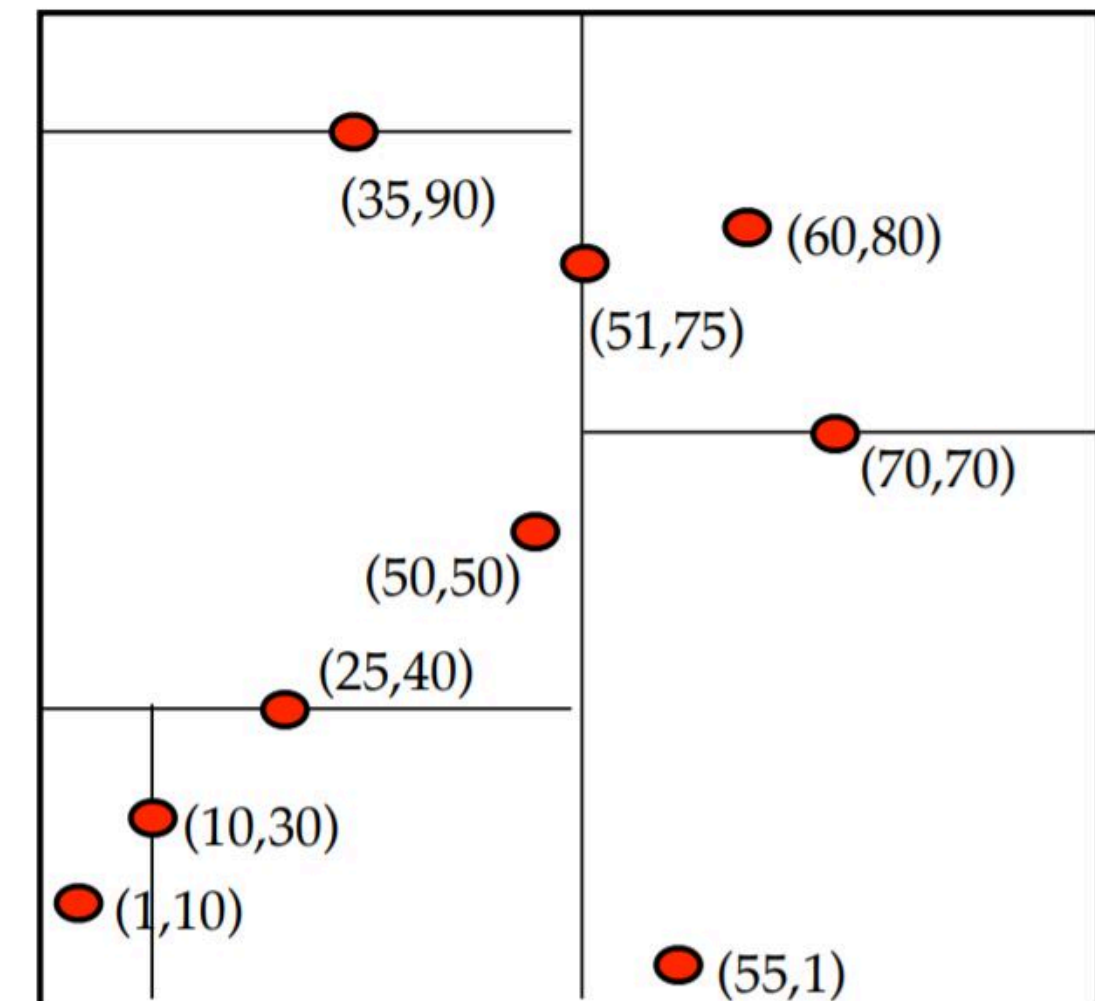
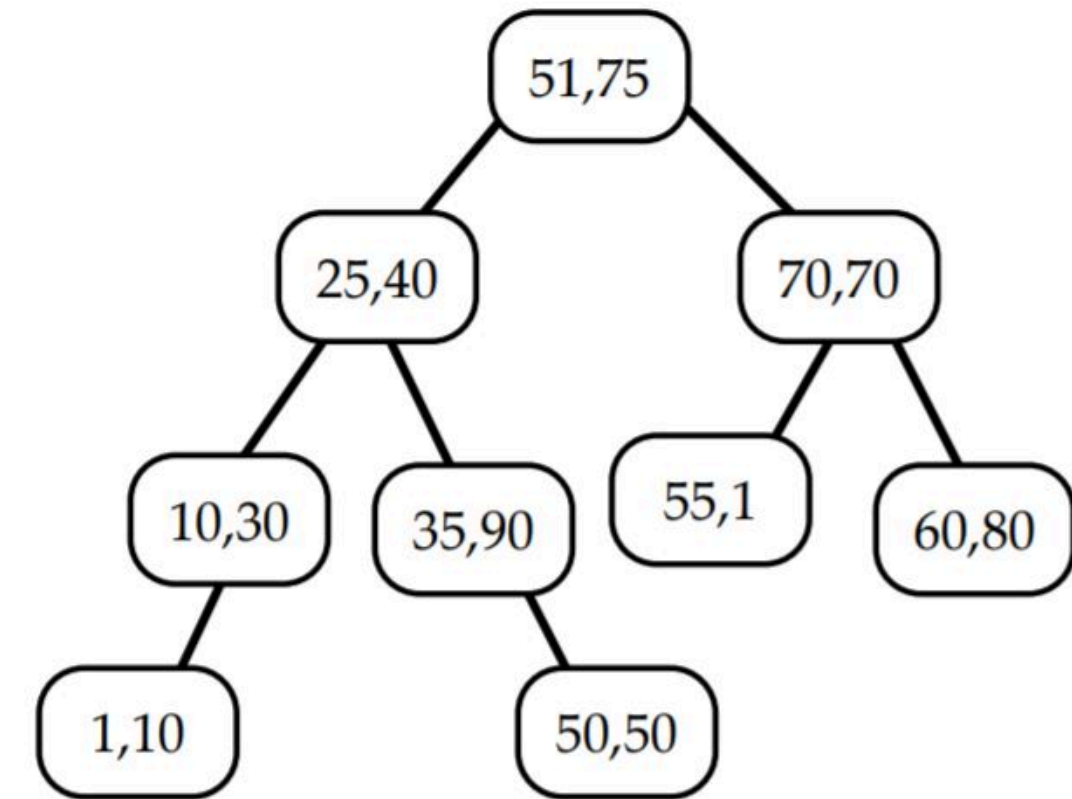
- 
- | | | | |
|---|--------------------|----|--------------------------------------|
| 1 | KD-Tree recap | 7 | Feature weighting |
| 2 | Problem | 8 | KD-Tree implementation |
| 3 | Goal | 9 | Recommendation search implementation |
| 4 | Implementation | 10 | Parameter variance analysis |
| 5 | Dataset | 11 | Interface |
| 6 | Data preprocessing | 12 | Evaluation |
| | | 13 | Conclusions |

KD-TREE RECAP

- Binary space partition
- Fast nearest neighbour search

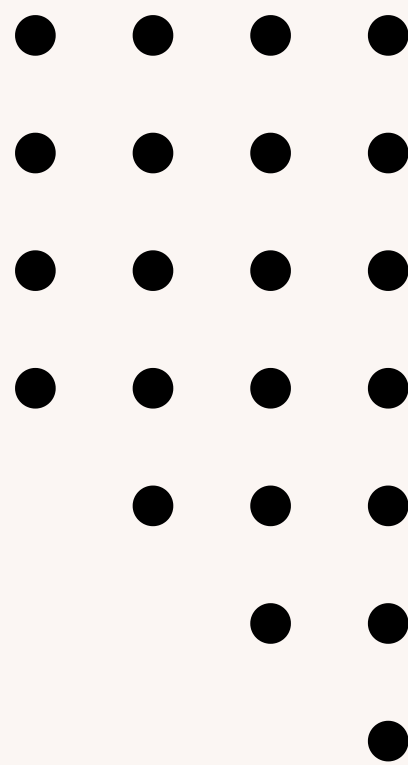
PROBLEM

- Online clothing shopping
- AI Recommendations
- Black box
- Add variance



GOAL

- KD-Tree solution
- Transparent recommendation system
- Fast and accurate recommendation retrieval



IMPLEMENTATION

- Dataset
- Data Preprocessing
- Feature Weighting
- KD-Tree Based kNN Search

DATASET

id	gender	masterCategory	subCategory	articleType
15970	Men	Apparel	Topwear	Shirts
39386	Men	Apparel	Bottomwear	Jeans
59263	Women	Accessories	Watches	Watches
21379	Men	Apparel	Bottomwear	Track Pants

id	baseColour	season	year	usage	ageGroup
15970	Navy Blue	Fall	2011	Casual	Adults-Men
39386	Blue	Summer	2012	Casual	Adults-Men
59263	Silver	Winter	2016	Formal	Adults-Women
21379	Black	Fall	2011	Casual	Adults-Men

id	price	discountedPrice	brandName	has_discount
15970	1195.0	1195.0	Turtle	0
39386	1499.0	1300.0	Peter England	1
59263	6500.0	6500.0	Titan	0
21379	1699.0	1699.0	Manchester United	0

DATA PREPROCESSING

- Price
- One Hot Encoding
- Color matching

FEATURE WEIGHTING

- Category
- subCategory
- Color
- Discount

$$d(x, y) = \sqrt{\sum_i w_i (x_i - y_i)^2}$$

KD-TREE IMPLEMENTATION

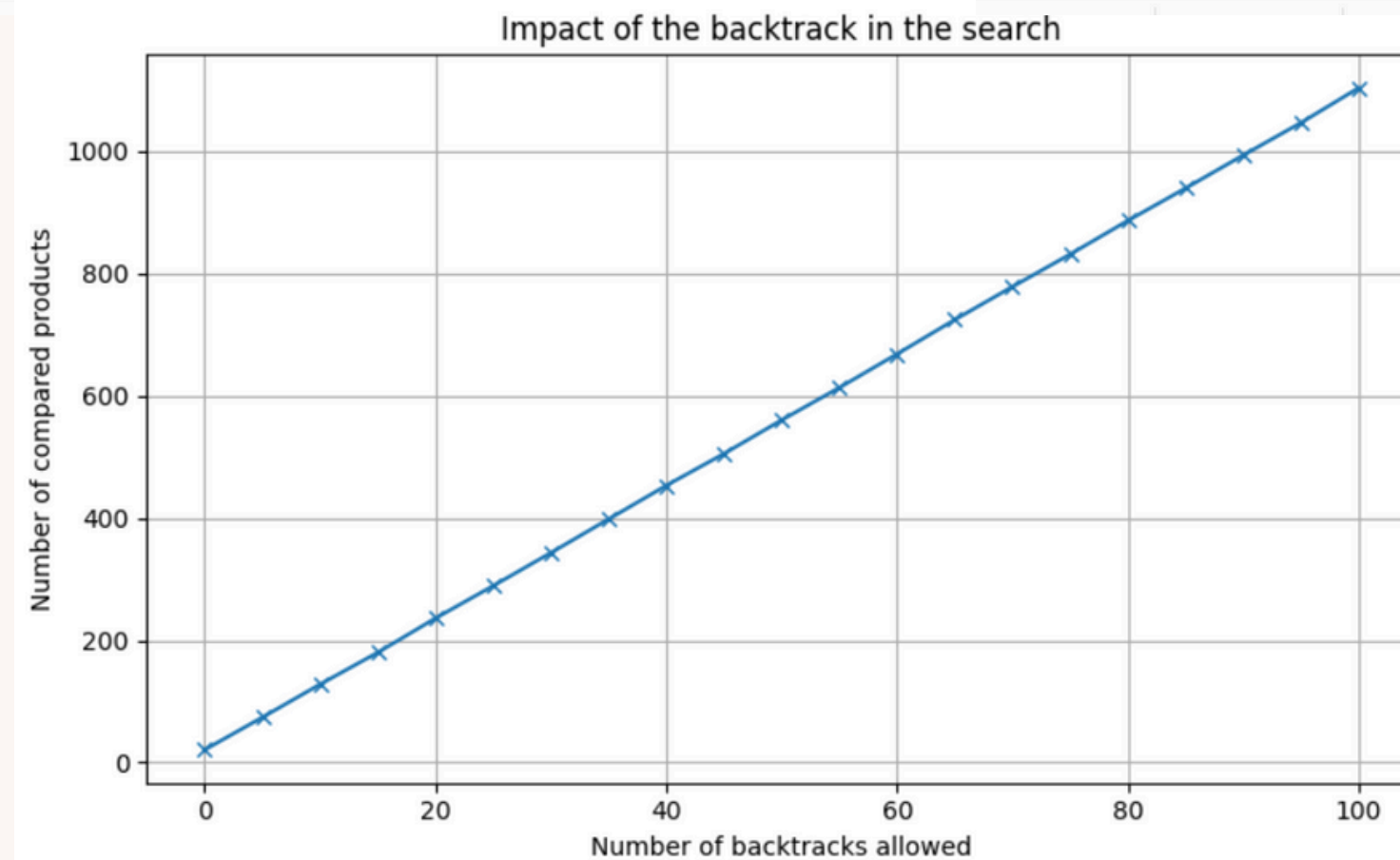
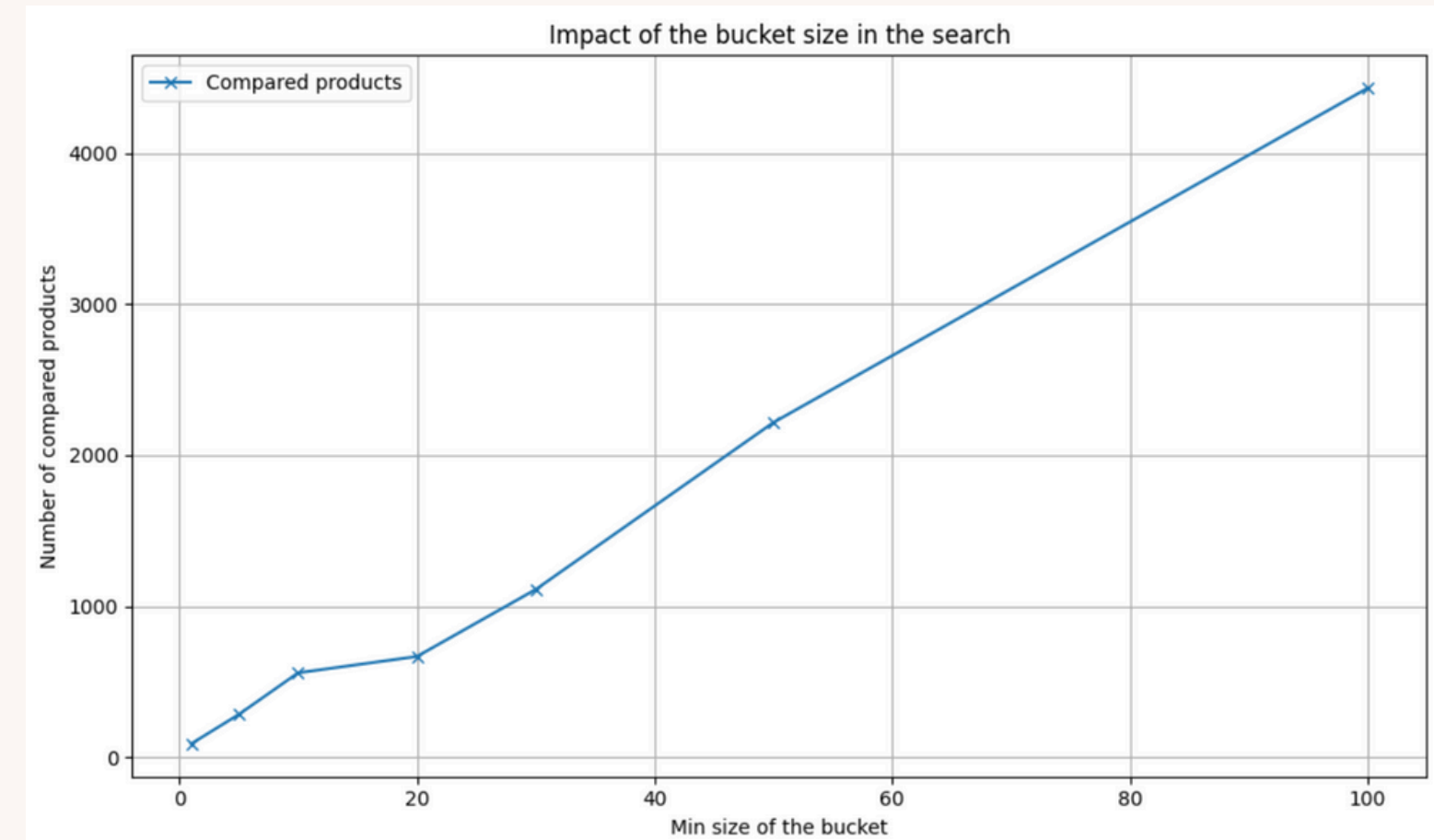
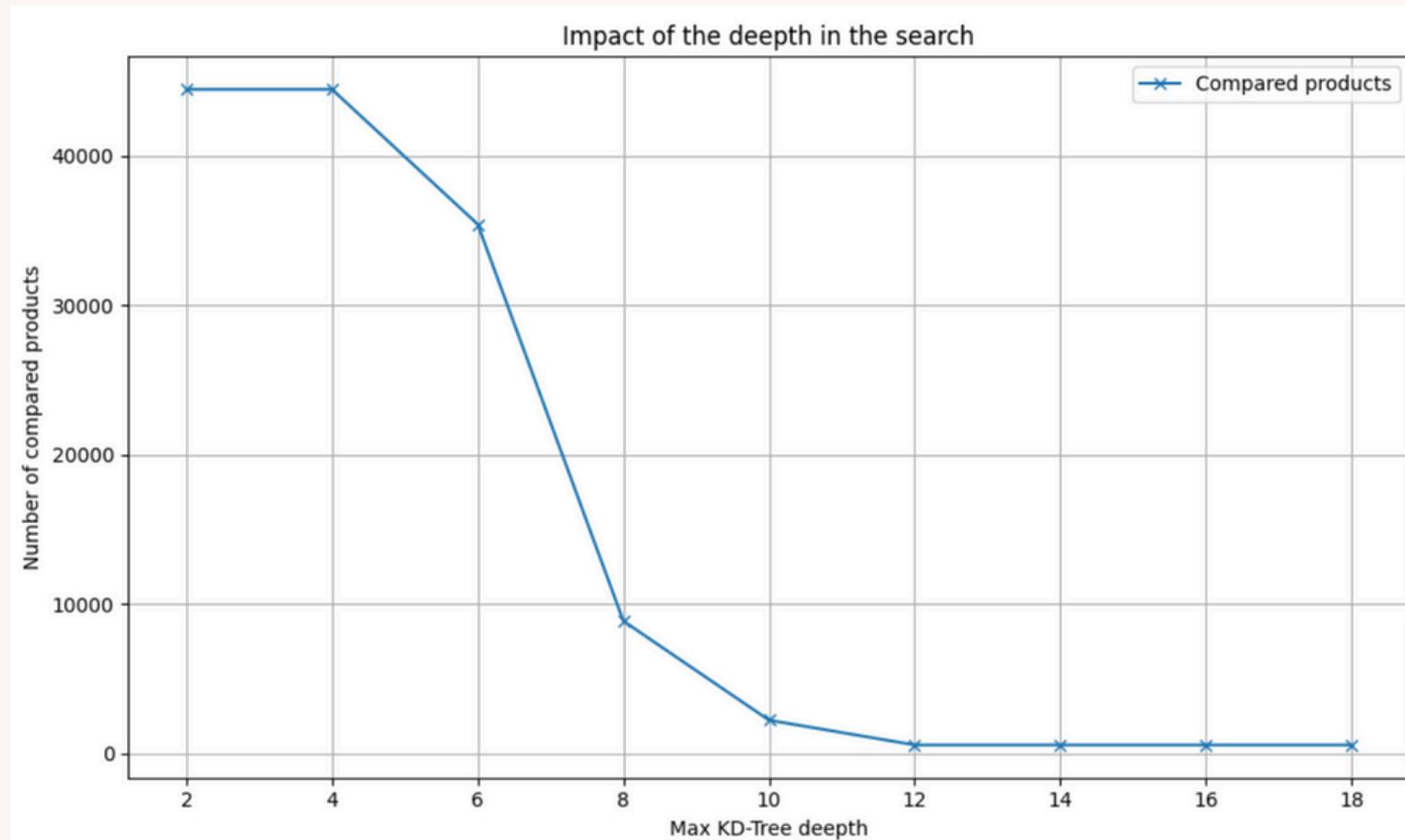
```
1 class KNode:
2     function __init__(points, axis=None, left=None, right=None):
3         self.points = points
4         self.axis = axis
5         self.left = left
6         self.right = right
```

```
1 def build_kdtree(points, depth = 0, split_axis_order = None, max_depth = 15, min_bucket_size = 10):
2
3     if depth >= max_depth or length(points) <= min_bucket_size:
4         return KNode(points, None)
5
6     axis = split_axis_order[depth]
7
8     left_points = [p for p in points if p[0][axis] == 0]
9     right_points = [p for p in points if p[0][axis] == 1]
10
11     return KNode(
12         points,
13         axis,
14         build_kdtree(left_points, depth + 1, split_axis_order, max_depth, min_bucket_size),
15         build_kdtree(right_points, depth + 1, split_axis_order, max_depth, min_bucket_size)
16     )
```

RECOMMENDATION SEARCH IMPLEMENTATION

```
1 function knn_search(root, target, k, weights, heap, number_of_backtracks):
2
3     if root.left is None and root.right is None:
4         for point, idx in root.points:
5             dist = sqrt(sum(weights * (target - point)^2))
6             heapq.heappush(heap, (-dist, idx, point))
7             if length(heap) > k:
8                 heapq.heappop(heap)
9         return sorted([(-d, idx) for d, idx, _ in heap])
10
11     axis = root.axis
12
13     if target[axis] == 0:
14         close_node = root.left
15         away_node = root.right
16     else:
17         close_node = root.right
18         away_node = root.left
19
20     knn_search(close_node, target, k, weights, heap, number_of_backtracks)
21
22     if length(heap) < k or number_of_backtracks[0] > 0:
23         number_of_backtracks[0] -= 1
24         knn_search(away_node, target, k, weights, heap, number_of_backtracks)
25
26     return sorted list of (-d, idx) from heap
```

PARAMETER VARIANCE ANALYSIS



INTERFACE



ID: 10000
¥324.0 (50% off)
¥649.0



ID: 10001
¥274.0 (50% off)
¥549.0



ID: 10002
¥274.0 (50% off)
¥549.0



ID: 10003
¥2695.0



ID: 10004
¥1995.0

Page 1 of 8884 [Next](#)

INTERFACE



ID: 10001
¥274 (50% off)
¥549



ID: 13516
¥299 (50% off)
¥599



ID: 9994
¥449 (55% off)
¥999



ID: 4906
¥299 (50% off)
¥599



ID: 4930
¥404 (55% off)
¥899

EVALUATION

Our system

product id	recommendations medium
1	0.7
2	0.7
3	0.6
4	0.4
5	0.5
6	0.5
7	0.5
8	0.6
9	0.8
10	0.2
11	0.9
12	0.2

SHEIN

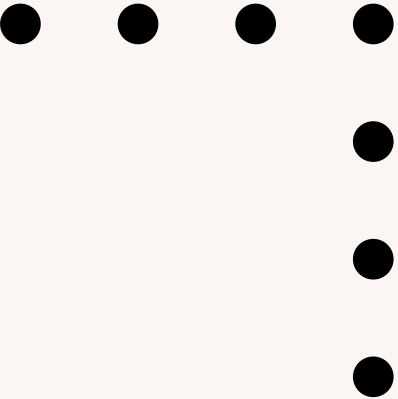
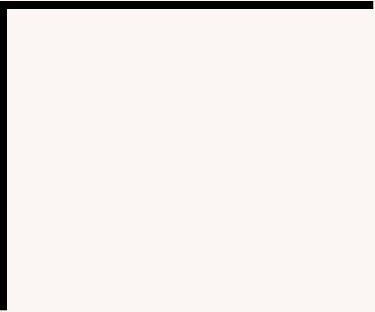
product id	recommendations medium
1	0.4
2	0.4
3	0.4
4	0.6
5	0.7
6	0.8
7	0.6
8	0.5
9	0.6
10	0.4
11	0.4
12	0.2

Primark

product id	recommendations medium
1	0.5
2	0.6
3	0.8
4	0.8
5	0.6
6	0.8
7	0.4
8	0.4
9	0.4
10	0.7
11	0.4
12	0.3

Results

system	medium performance
Our system	0.550
SHEIN's system	0.500
Primark's system	0.558



CONCLUSIONS

- Transparency of the results
- Testing different weight combinations
- Assigning more importance to the most common characteristics users look